CpSc 418 **Homework 5** Due: Nov. 14, 2025, 11:59pm

Early Bird: Nov. 12, 2025, 11:59pm

142 points +1 extra credit)

Please submit your solution using handin:

```
handin cs-418 hw5
```

Your solution should have three files file: `hw5.pdf`.

The Questions

0. **Collaborators** (**2 points (+1 extra credit)**).
   In your `hw4.pdf` list anyone you collaborated with or outside materials that you used.

   - If the collaborator is a student in this class, give their CWL.
   - If the colloborator is a human who is not in this class, give their name and a short (i.e. one sentence) description of their relevance to this course.
   - If you used on-line material, give the URL.
   - If you used an AI assistant, give the name of the assistant. We'll give extra credit if you include your prompt, and one sentence to say what was useful in the response.
   - Other sources such as books should be just cited clearly.

   For each collaborator, list what questions you collaborated on.

   You do not need to list course materials including anything on the cs-418 website, assigned readings, the official Erlang documentation, or material from office hours or tutorials.

   If you had no collaborators, write "*No collaborators*". A blank or omitted response will get no credit.

   Extra Credit: When grading HW 1, one of our TA's pointed out that many of you did an extra fantastic job on this question providing detailed prompts that you used with AI assistants. We really appreciate this as we are learning how we can best leverage emerging technologies in our teaching. In fact, we appreciated it so much that we gave a point of extra credit for answers that the TA flagged as extra good! Thanks.

   Going forward, we want to make this opportunity to everyone, whether or not you use AI. Here are *five* ways you can get that extra credit point:

   (a) Include your prompts to AI assistants as described above.

   (b) Tell us why you didn't use an AI assistant or didn't find it helpful.

   (c) Write a few sentences that you think would be the best advice you could give to a student starting this assignment based on your experience of completing it.

   (d) Identify a question that you thought was particularly helpful or pointlessly tedious, and give us a few sentences saying why.

   (e) Write something else that you think is worthy of extra-credit. Mark it clearly as your response to the **Extra Credit** part of this question.

1. Branch Prediction: (**35 points**)
   Superscalar processors can execute multiple instructions in a single clock cycles. Typical programs have branches every 5 to 10 instructions, and processors tend to have deep execution pipelines; it can take five to ten or more clock cycles from fetching an instruction until its result is available. Conditional and computed branches are a headache for computer architects because we don't know what instruction to fetch next until the branch instruction completes. For example, if a branch takes five clock cycles to

complete, and the superscalar processor fetches four instructions per clock cycles, *twenty* instructions will be fetched between fetching a branch and knowing its outcome. This likely included a few more branches!

To address these issues, superscalar processors use branch prediction. A simple predictor simply counts the number of times the branch is taken or not. The counter is incremented if the branch is taken, and decremented when it is not taken. For example, we could have a three bit counter that saturates at `+3` or `-4`. If a branch is taken, and the counter is already at `+3`, it stays at `+3`. Likewise for decrementing the counter when the branch is not taken. When the branch is fetched, if the branch is predicted to be taken, the processor jumps to fetching from the branch target, and if the branch is predicted as untaken, the processor continues fetching the instructions immediately after the branch. See the reading on superscalar processors or the Oct. 6 slides for more details.

This problem explores the cost of branch misprediction. The key function for this is `ar_op` in `array.c`. If we look at the assembly code for the `ar_op` function, we find a conditional branch corresponding to the `i < n-7` test of the `for`-loop, and a computed branch corresponding to the `switch` statement.

(a) (**10 points**) Assume that the branch corresponding to `i < n-7` is *taken* when `i < n-7` and not taken when `i >= n-7`. This is a slight simplification; in the actual assembly code, there is one branch at the beginning of the `ar_op` function to make sure `n-7 >= 0`, and then a branch at the end of the for-loop body to branch back to the start of the loop body if `i < n-y`.

With the assumptions of described in the previous paragraph, consider a call to `ar_op(op, n)` `n > 7`.

i. (**2 points**) How many times in total is the branch taken for this call to `ar_op`?
n-7 times.

ii. (**2 points**) How many times in total is the branch not-taken for this call to `ar_op`?
1 time.

iii. (**6 points**) If the branch-predictor uses the saturating counter method described above, what fraction of the `i < n-7` branches will be predicted correctly?
Provide a brief (one or two sentences) justification for your answer.

$$\begin{cases} \frac{0}{1} & \text{if } n \leq 7 \\ \frac{n-7}{n-6}, & \text{otherwise} \end{cases}$$

Since the incrementor starts at 0 and increases to 3, it will only be incorrect one time and that is when $i = n - 7$. So it will correctly predict $n - 7$ out of $n - 6$ branches.

(b) (**5 points**) Consider the branch corresponding to the switch statement, i.e. the branch that can go to one of eight `case`s depending on the value of `op[i + (acc & y)]`. For simplicity, assume that the branch predictor guesses that the branch goes to same place as the previous time the branch was executed.

i. (**2 points**) If all elements of `op` are the same, for example `op[i] = 0` for all `0 <= i < n`, what fraction of the `op[i + (acc & y)]` branches will be predicted correctly?
$\frac{n-8}{n-7}$ assuming the first branch mispredicts (most common case), or 1 if the first branch happens to predict correctly by chance.
Since all elements of `op` are the same, the switch statement always goes to the same case. The predictor will mispredict on the first iteration (no history), but correctly predict all subsequent $n - 8$ iterations.

ii. (**3 points**) If the elements of `op` are uniformly random with `0 <= op[i] < 8` for all `0 <= i < n`, what fraction of the `op[i + (acc & y)]` branches will be predicted correctly?
Note: this is only an approximation because there is some correlation of `acc & 7` and `prev & 7`. We will ignore that.

$\frac{1}{8}$

Since `acc` is effectively random and `acc & 7` produces values from 0 to 7, we access uniformly random elements of `op`. Each element randomly selects one of 8 cases. The predictor assumes the same case as the previous iteration, giving a $\frac{1}{8}$ chance of a correct prediction.

(c) (**15 points**) Compile and run `op` on `thetis.students.cs.ubc.ca` or `gambier.student.cs.ubc.ca`. The command

```
./op 1000 100 10000 const0
```

will give the mean and standard deviation for the time to execute `ar_rand` with `n=1000` and randomly distributed elements for `op`. In a bit more detail, the command is

```
./op n_op ntrials runs_per_trial dist
```

where

`n_op`: number of elements in the `op` array.

`n_trials`: Perform `ntrials` timing measurements. Report the mean and standard deviation.

`runs_per_trial`: Each timing measurement consists of calling `ar_op(op, n_op)` `runs_per_trial` times. The total time for each measurement is dividdd by `runs_per_trial` to get a value for the time per call to `ar_op`. This is because the time for an individual call is short enough to introduce large clock quantization errors.

`dist`: The distribution to use for the elements of `op`. `dist="rand"` generates random elements with each element in `0`, ..., `7`. `dist="constN"` where `N` is a decimal digit sets each element to `N`. `"c"` is an abbreviation for `"const0"`.

i. (**5 points**) Include a table of your raw timing data for your answers to this the remaining parts of this question.

Distribution	n_op	Mean Time (s)	Std Dev (s)
const0	1000	1.4307×10^{-6}	1.1703×10^{-7}
rand	1000	6.7281×10^{-6}	2.0516×10^{-6}

Parameters: n_trials=100, runs_per_trial=10000, executed on thetis.students.cs.ubc.ca

ii. (**5 points**) By running `./op` with random and constant distributions for the elements of `op`, determine the branch mispredict penalty, in nanoseconds, for executions on `gambier` or `thetis`.

Total time difference: $(6.7281 - 1.4307) \times 10^{-6} = 5.2974 \times 10^{-6}$ seconds $= 5297.4$ nanoseconds
Number of loop iterations: $n - 7 = 1000 - 7 = 993$
Mispredictions with const0: ≈ 1 (first iteration only)
Mispredictions with rand: $993 \times \frac{7}{8} \approx 869$
Additional mispredictions with rand: $869 - 1 = 868$
Branch mispredict penalty: $\frac{5297.4 \text{ ns}}{868} \approx$ **6.1 nanoseconds**

iii. (**5 points**) The Intel Xeon E5-2698 v3 processors in `thetis` and `gambier` have a base CPU clock frequency of 2.3GHz and a boost-clock frequency of 3.6GHz. How many clock-cycles is the branch-mispredict penalty if the CPU is operating at the base clock? How many clock-cycles is the branch-mispredict penalty if the CPU is operating at the boost clock? If `thetis` or `gambier` is lightly loaded, you should see the boost-clock performance, but I'm not absolutely certain of that.

base clock: $(\frac{6.1ns}{10^9 ns/s}) \cdot (2.3 \cdot 10^9 cycles/s) = 14.03 \approx 14$ cycle branch-mispredict penalty
boost clock: $(\frac{6.1ns}{10^9 ns/s}) \cdot (3.6 \cdot 10^9 cycles/s) = 21.96 \approx 22$ cycle branch-mispredict penalty

(d) (**5 points**) I count 14 to 18 x86 instructions per iteration of the `for(i=0; ...)` loop in `ar_op`. Let's take the midpoint of 16 instructions per iteration. Assuming the boost clock rate of 3.6GHz, how many instructions does the `thetis` or `gambier` execute per clock cycle based on your data above?

```
int ar_op(int *op, int n) {
    int prev = 0, acc = 0, next = 0;
    for(int i = 0; i < n-7; i++) {
        switch (op[i + (acc & 7)]) {
            case 0:  next = prev + acc; break;
            case 1:  next = prev - acc; break;
            case 2:  next = prev & acc; break;
            case 3:  next = prev | acc; break;
            case 4:  next = prev ^ acc; break;
            case 5:  next = acc + 0x1357; break;
            case 6:  next = acc ^ 0x1357; break;
            case 7:  next = acc << 11; break;
            }
        prev = acc;
        acc = next;
        }
    return(acc);
    }
```

Figure 1: The `ar_op` function for Question 1

Note: the number that you get from this estimate is probably a bit high. The x86 internally translates x86 instructions into "simpler", ROPs, roughly RISC instructions. In the process, it appears to do some local optimizations. For example, I expect that it figures out how eliminate some of the inefficiencies that occur because the x86 instruct set only has eight programmer visible registers. The ROP count might be closer to 9 to 12 operations per loop iteration, but I don't know of a way to check the details.

Number of loop iterations: $n - 7 = 1000 - 7 = 993$
993 loops * 16 instructions / loop = 15888 total instructions
const: $\frac{15888 ins}{(3.6 \cdot 10^9 cycles/s) \cdot 1.4307 \times 10^{-6} s} = 3.085$ ins / cycle
rand: $\frac{15888 ins}{(3.6 \cdot 10^9 cycles/s) \cdot 6.7281 \times 10^{-6} s} = 0.656$ ins / cycle

2. Dynamic Programming (**37 points**) For Homework 4, you examined the performance of dynamic programming using the work-span model. The work-span approach gives shows the dependencies in dynamic programming which indicates how one *could* make a parallel implementation. The work-span model ignores communication cost. To obtain an *efficient* implementation, we need to find a way to assign tasks to processors and account for the communication costs.

   One approach to implementing a parallel version of dynamic programming is to partition the tableau into "tiles". If each tile has N_{tr} rows and N_{tc} columns, then a process will perform $N_{tr}N_{tc}$ work before needing to communicate with adjacent tiles. This allows us to amortize the communication costs.

   (a) Implement a parallel version of dynamic programming for the editing-distance problem (**15 points**).
   The file hw5.erl provides template code, and hw5_lib.erl provides:

   - A sequential implementation of dynamic programming for your reference. This is also useful for tests. Some of you may find it to be a useful example if you need to review dynamic programming.

- Functions for handling a single "tile" of the dynamic programming tableau. That's right – we've given you most of the sequential code that you need, you just need to make it parallel.
- Functions for working with chains of processes. That's right – we're even giving you the low-level parallel code. My implementation uses such a chain.
- Why chains of processes? Think about the work-span model for the parallelism available in dynamic programming that you worked out in Homework 5. There is an "obvious" way to write a parallel version that uses one process for each tile, but this is very inefficient because most of the processes are idle most of the time. What is the maximum number of processes that can be active at one time? Call this $P_{\max}$. Can you implement your parallel version using $P_{\max}$ processes?

The `hw5:ed_par` function takes a parameter of `TileWidth`. That might be specific to my implementation. If you need different parameters for configuring your implementation, please make a private post to piazza. We'll work it out.

(b) Write a one to three paragraph describing your parallel implementation (**5 points**).
This should be sufficient to make it easy for another programmer to understand your solution. You can see the comments in hw5_lib.erl as an example. In fact, more than half of the lines in hw5_lib.erl are comments. You may not need to be that thorough, but you should make your design choices clear.

(c) Write some test-cases for `hw5:ed_par`. (**5 points**)
Feel free to use `hw5_lib:ed_seq` as a reference. If you used `hw5_lib:default_op_costs()`, round-off error shouldn't affect your results, and that will make comparisons easier.
If you wrote other functions, provide tests for them as well.

(d) Measure and report speed-up. (**12 points**)
As usual, please run your timing measurements on `gambier.students.cs.ubc.ca` or `thetis.students.cs.ubc.ca`. Use test cases where the time for `hw5_lib:ed_seq` is up to one second. I found that I could have strings of 1000 to 2000 characters and satisfy this constraint.

i. Include your raw data. (**4 points**)

ii. Show how your choices of configuration parameters impact the result. (**3 points**)
For my solution, there is some impact on changing the number of processes or the tile width, but the region of optimal performance is pretty flat.

iii. Report the best speed-up you could get (**2 points**)
Note: I got a speed-up of about 8 on `thetis` with 32, 2-way multithreaded cores. Running on my laptop (M1-Pro CPU, 8 cores), I get a speed-up of about 5.

iv. Identify causes of performance loss for this example? (**3 points**)
You may need to speculate, but give some evidence for your guesses.

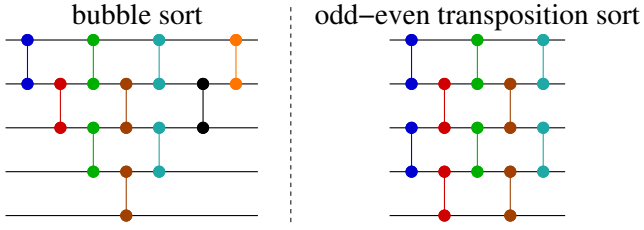

Figure 2: Sorting networks for a linear array

.

3. Sorting on a linear array. (**20 points**)
Figure 2 shows two sorting networks for sorting and array of five elements. These can be implemented on a linear-array of processors, and each compare-and-swap element only compares two adjacent elements in the network. Of course, they can be implmented on higher-dimensional networks such as meshes, hypercubes, or the Dragonfly; they don't *need* all of the connectivity that those fancier networks provide. The bubble-sort and odd-even transposition sorting networks can be generalize for an arbitrary number of inputs, N as seen in the `hw5:bubble/1` and `hw5:odd_even/1`; see `hw5.erl`.

   (a) (**4 points**): What are the work and span for the bubble-sort and odd-even transposition sort networks with five inputs as shown in Figure 2:

      i. (**1 point**) Work for `bubble(5)`:
      10

      ii. (**1 point**) Span for `bubble(5)`:
      7

      iii. (**1 point**) Work for `odd_even(5)`:
      10

      iv. (**1 point**) Span for `odd_even(5)`:
      5

   Only count `compare_and_swap` operations and count each operation as contributing 1 to the work or span.

   (b) (**8 points**): What are the work and span for the bubble-sort and odd-even transposition sort networks with `N` inputs:

      i. (**2 points**) What is the work for `bubble(N)`?
      $\frac{n(n-1)}{2}$

      ii. (**2 points**) What is the span for `bubble(N)`?
      2n - 3

      iii. (**2 points**) What is the work for `odd_even(N)`?
      $\frac{n(n-1)}{2}$

      iv. (**2 points**) What is the span for `odd_even(N)`?
      n

   Hint: if you prefer coding to pencil-and-paper derivation, you can get a lot of "hints" by experimenting with the code in `hw5.erl`. Once you "know" the answer from those experiments, writing a short justification should be fairly easy.

   (c) Comparing bubble-sort and odd-even transposition sort (**4 points**):

      i. (**2 points**): What is $\lim_{\texttt{N}\to\infty} \frac{\mathsf{Work}(\texttt{bubbleN})}{\mathsf{Work}(\texttt{odd\_evenN})}$
      1

      ii. (**2 points**): What is $\lim_{\texttt{N}\to\infty} \frac{\mathsf{Span}(\texttt{bubbleN})}{\mathsf{Span}(\texttt{odd\_evenN})}$
      2

   In class, we gave an informal argument for the correctness of the bubble-sort network, In the remaining parts of this question, you will prove the correctness of odd-even transposition sort.

   Let x_j, be the input to the j^{th} column of compare-and-swap elements. We are using zero-based indexing, so x_0 is the input to the sorting network. The output of the sorting network is $x_{\texttt{Span\_oe(N)}}$ where `Span_oe(N)` is the span for `odd_even(N)`, i.e. your answer to Question 3(b)iv. We write $x_j[i]$ to denote i^{th} element of x_j, again using zero-based indexing. To avoid special cases, we will define $x_j[-1] = -\infty$, and $x_j[N] = +\infty$ for all $j \in \{0, \ldots, \texttt{Span(N)}$. We write $\mathrm{sort}(x_j)$ to denote the result of sorting x_j. Because sorting network neither delete nor duplicate elements,

$$\mathrm{sort}(x_1) \;=\; \mathrm{sort}(x_1) \;=\; \ldots \;=\; \mathrm{sort}(x_{\texttt{Span\_oe(N)}}) \;=\; \ldots \;=$$

From these definitions we get:

$$x_j[i] \quad = \quad \begin{cases} \min(x_{j-1}[i], x_{j-i}[i+1]), & \text{if } (i \bmod 2) = (j \bmod 2) \\ \max(x_{j-1}[i], x_{j-i}[i-1]), & \text{if } (i \bmod 2) \neq (j \bmod 2) \end{cases}$$

For example, when `N=5`:

$j=1$	$j=2$	$j=3$	$\ldots$
$x_1[0] = \min(x_0[0], x_0[1])$	$x_2[0] = \max(x_1[-1], x_1[0])$ $= \max(-\infty, x_1[0])$ $= x_1[0])$	$x_3[0] = \min(x_2[0], x_2[1])$	$\ldots$
$x_1[1] = \max(x_0[0], x_0[1])$	$x_2[1] = \min(x_1[1], x_1[2])$	$x_3[1] = \max(x_2[0], x_2[1])$	$\ldots$
$x_1[2] = \min(x_0[2], x_0[3])$	$x_2[2] = \max(x_1[1], x_1[2])$	$x_3[2] = \min(x_2[2], x_2[3])$	$\ldots$
$x_1[3] = \max(x_0[2], x_0[3])$	$x_2[3] = \min(x_1[3], x_1[4])$	$x_3[3] = \max(x_2[2], x_2[3])$	$\ldots$
$x_1[4] = \min(x_0[4], x_0[5])$ $= \min(x_0[4], +\infty)$ $= x_0[4]$	$x_2[4] = \max(x_1[3], x_1[4])$	$x_3[4] = \min(x_2[4], x_2[5])$ $= \min(x_2[4], +\infty)$ $= x_2[4]$	$\ldots$

Note: `lists:nth(j, hw5:odd_even(N))` lists the compare-and-swap elements as `{k,`ℓ`}` pairs to indicate that inputs are from lines `k` and ℓ; the `min` output goes to line `k` and the `max` output goes to line ℓ. Where $1 < j \leq$ `Span_oe(N)`; $0 \leq k < N$; and $0 \leq \ell < N$. You can give Erlang the command

```
> lists:nth(1, hw5:odd_even(5)).
[{0,1},{2,3}]
```

to see the compare-and-swap elements that output x_1, and `lists:nth(J, hw5:odd_even(5))` to see the compare-and-swap elements that output x_J.

Because we are reasoning about sorting networks, we only need to consider inputs that consist entirely of `0`s and `1`s. We will assume that x_0 is such an input in the remainder of this problem. Let

$$\text{count1}(x_j, i) \quad = \quad \sum_{k=0}^{i-1} x_j[k]$$

In English, $\text{count1}(x_j, i)$ counts the number of `1`s that occur in x_j *before* element `i`. Note that if x_j is sorted, then all of the `0`s in x_j appear at the beginning. This implies that x_j is sorted iff

$$\begin{array}{l} \forall 0 \leq i < N. \\ \quad (x_j[i] = 0) \Rightarrow (\text{count1}(x_j, i) = 0) \end{array}$$

Furthermore, if $x_j[i] = 0$, then

$$\text{sort}(x_j, i - \text{count1}(x_j, i)) \quad = \quad 0$$

In the following, you will compute a bound for j that ensures all of the `0`s of x_0 have been "shuffled" to be to the left of all of the `1`s.

We will start by considering an input x_0 such that for some i, $\text{count1}(x_0, i) = 1$. For example, if $N = 8$, and $x_0 = [0, 0, 1, 0, 0, 0, 0, 0]$, then $\text{count1}(x_0, i) = 1$ for any i with $3 \leq i < 8$. I'll use the function `hw5:sortv(Data)` to show the outputs of each column of compare-and-swap elements:

$J = 0$: |0,0|1,0|0,0|0,0|
$j = 1$: 0|0,0|1,0|0,0|0
$j = 2$: |0,0|0,0|1,0|0,0|

$j = 3$: 0|0,0|0,0|1,0|0
$j = 4$: |0,0|0,0|0,0|1,0|
$j = 5$: 0|0,0|0,0|0,0|1
$j = 6$: |0,0|0,0|0,0|0,1|
$j = 7$: 0|0,0|0,0|0,0|1

The code outputs | as a separator to highlight pairs of elements of x_j that will be compared-and-swapped to produce x_{j+1}. I have highlighted the 1 at $x_0[2]$ and the 0 at $x_0[6]$ and shown how they propagate during the sort. Observe that the 1 that started at $x_0[2]$ and the 0 that started at $x_0[6]$ are exchanged to produce x_4. In other words, $j = 4$ is the smallest value of j such that $\text{count1}(x_j, 6) = 0$.

We note that if x_0 is all 0s or all 1s, then it is already sorted. We will now consider the less trivial cases.

(d) (**5 points**)
Let x_0 be an input that has exactly one 0 – all other elements of x_0 are 1s. Let i_0 be the index of that 0 in x_0; in other words, $x_0[i_0] = 0$, and $x_0[i] = 1$ for all $i \neq i_0$. For $0 \leq j$, let i_j be the index of the 0 element in x_j. Show that:

$$\begin{array}{rcll} i_j & = & i_0 + 1 - j, & \text{if } i_0 \text{ is even and } j \leq i_0 + 1, \text{ and } j > 0 \\ i_j & = & i_0 - j, & \text{if } i_0 \text{ is odd and } j \leq i_0 + 1 \\ i_j & = & 0, & \text{otherwise.} \end{array}$$

Note that this shows that for inputs with one 0 the Data is sorted after at most $N - 1$ steps if N is even, and N steps if N is odd.
Hints: My proof is a simple proof by induction on j. The base case is $j = 0$, for which the claim is trivial. I use the definition of the odd-event transposition sorting network:

- When j is even, for each *even* integer i with $0 \leq i < N$,

$$\begin{array}{rcl} x_{j+1}[i] & = & \min(x_j[i], x_j[i+1]) \\ x_{j+1}[i+1] & = & \max(x_j[i], x_j[i+1]) \end{array}$$

- When j is even, for each *odd* integer i with $0 \leq i < N$,

$$\begin{array}{rcl} x_{j+1}[i] & = & \min(x_j[i], x_j[i+1]) \\ x_{j+1}[i+1] & = & \max(x_j[i], x_j[i+1]) \end{array}$$

If you need more hints, run `hw5:sortv(Data)` trying various lists for `Data` with one `0` and the rest `1s`. And then drop by office hours or post questions to piazza.

(e) (**10 points**)
Let x_0 be an input of N to an odd-even transposition sorting network. Let $N_0(x)$ be the number of 0s in x. Noting that $N_0(x_0) = N_0(x_1) = \ldots$, we will simply write N_0 when the x_j sequence is clear from context. For $0 \leq k < N_0$ let $i_{j,k}$ be the position of the k^{th} zero in x_j. For example, if

$$x_u = [0, 1, 1, 0, 1, 1, 1, 0]$$

then $N = 8$, $N_0(x_j) = 3$, $i_{j,0} = 0$, $i_{j,1} = 3$, and $i_{j,2} = 7$. We note that x_j is sorted if for all k with $0 \leq k < N_0$, $i_{j,k} = k$.
$i_{0,k}$ is defined from the input data.

$$\begin{array}{rcll} i_{j,0} & = & i_{j,0} + 1 - j, & \text{if } i_{j,0} \text{ is even and } j \leq i_{j,0} + 1, \text{ and } j > 0 \\ i_{j,0} & = & i_{j,0} - j, & \text{if } i_{j,0} \text{ is odd and } j \leq i_{j,0} + 1 \\ i_{j,0} & = & 0, & \text{otherwise.} \end{array}$$

For $j > 0$ and $k > 0$ prove that

$$\begin{array}{rcll} i_{j,k} & = & i_{j-1,k} - 1, & \text{if } (k \bmod 2) = (j \bmod 2) \text{ and } i_{j-1,k-1} < i_{j-k,k} - 1 \\ i_{j,k} & = & i_{j-1,k}, & \text{otherwise} \end{array}$$

Notes: the condition that $(k \bmod 2) = (j \bmod 2)$ says that if k is even, then $j-1$ is odd, or vice-versa. This means that $x_{j-1}[i_k]$ is in the upper position for the compare-and-swap and can be exchanged with $x_{j-1}[i_k - 1]$ if $x_{j-1}[i_k - 1]$ is a `1`. The condition that $i_{j-1,k-1} < i_{j-1,k} - 1$ means that there is a gap between the `0` at position $i_{j-1,k-1}$ and the `0` at position $i_{j-1,k}$. Therefore, there is a `1` between in position $i_{j-1,k} - 1$. **Hints:**

- Try `hw5:sortv(Data)` for many choices of `Data` that are lists `0`s and `1`s. You may find it helpful to screenshot or print the output, and then trace the path of each `1` from the first line of the output to the last.
- As usual, office hours and tutorials are good.
- Post questions to piazza. I suspect this question might need a few more hints. If you bring questions to office hour, tutorials, or piazza, I'll try to think of more hints.

(f) (**5 points**)

4. Hénon Maps. **48 points**

The Hénon map is a two variable recurrence defined below:

$$\begin{array}{rcl} x_{i+1} & = & 1 - ax_i^2 + y_i \\ y_{i+1} & = & bx_i \end{array} \tag{1}$$

For this problem, we choose $a = 1.3$ and $b = 0.3$. The Hénon map is a chaotic dynamical system: starting from two points that are initially close, the sequences will diverge from each other. On the other hand, for initial points that aren't "too large", all subsequent points stay in a bounded region. For our use, the Hénon map provides a computation that is purely floating-point arithmetic bound – the SMs can hold all of the values that they need in registers. We use this as a way to see how fast a GPU can perform floating point operations.

The file henon.cu provides CPU and GPU implementations of the Hénon map. You can build the executable `henon` with the command

```
make henon
```

given that you've copied all of the source files into your current directory. The provided Makefile is configured for use on the `linXX.students.cs.ubc.ca` machines. Once you have an executable, you can run it with the command:

```
./henon n_blocks threads_per_block n_iter n_trials elem_size
```

Each of these has a default value provided. If you only give some of the arguments, the remaining ones will get their default values. Likewise, If you give a single "`-`" as an argument, that parameter will get it's default value. The defaults are (currently):

`n_blocks`= 100, `threads_per_block`= 1024, `n_iter`= 1000000, `n_trials`= 5, and `elem_size`= 4.

The code generates `n_blocks*threads_per_block` random initial points for the Hénon map, and performs `n_iter` iterations on each of these points using the GPU. The parameter `elem_size` specifies whether the kernal uses single-precision (when `elem_size==4`) or double-precision (when `elem_size==8`) arithmetic. I had intended to include a half-precision case with `elem_size==2`, but my half-precision code ran *slower* than the single-precsion version. I need to figure that out before I inflict such code on the class.

The time for the CUDA kernel execution is measured. This is repeated `n_trials` times, and the mean and standard deviation of the execution time are reported. We also report the mean and standard deviation of the square of the step-size for the iteration, where

$$step_i^2 = (x_i - x_{i-1})^2 + (y_i - y_{i-1})^2$$

This should be robust even though the Hénon map sequence is chaotic.

With that out of the way, here are the questions:

(a) **(10 points)** Complete the body of the `__global__` function `henon32`.
Hints:

- This is *really* easy. Most of the code can copied from the body of `henon_cpu`.
- You will need to compute this thread's index for accessing the arrays `x0`, `y0`, `s2`, and `s4`. You can look at other GPU kernels, for example in examples.cu to figure out what you need.
- Likewise, you will need to check to make sure that the index for this thread is in bounds. Again, you can look the kernels in examples.cu to see examples.
- The GPU and CPU versions won't produce the exact same answers, because C (running on the x86) converts all `float`s to `double`s when doing arithmetic, and then rounds the final value when assigning to a `float`. The GPU code does everything with `float`s if the operands are `float`s. The rounding error makes the two sequences of points diverge because the Hénon map is chaotic. The good news is that both versions converge to the same estimates of RMS step size and variance. I might find some other ways you can test your code – or you can.

(b) **(3 points)** The GPUs on the `linXX.students.cs.ubc.ca` machines have nVidia RTX 3070 Ti GPUs. How many SMs does the RTX 3070 Ti GPU have? Cite your source. Better yet, run the program `cuprop` (built from `cuprop.cu` in the source directory for this assignment. Either way, explain how you got your answer.

48 SMs

Sources:
NVIDIA developer forum
Wikipedia NVIDIA gpu spec
Confirmed with cuprop's multiProcessorCount which lists 48.

(c) **(10 points)** Experiment with different choices for `n_blocks` to find a choice that maximizes the number of floating point operations performed per second with the reported mean for the execution time being less that 0.5 seconds. For simplicity, use `threads_per_block=1024` and `n_iter=1000000` for all of your trials. What is the maximum number of floating point operations per second that you were able to obtain?

Make your timing measurements using one of the `linXX` machines. Please state which machine, and use the same machine for all questions about the Hénon map. Include a table with your measurements in your solution.

I also tried other values for `threads_per_block` and `n_iter`. While there are other values that achieve maximize performance to the same value as with `threads_per_block=1024` (within measurement error), I didn't find any that were faster. Feel free to try your own experiments.

n_blocks	n_data	Mean Time (s)	GFLOPS
10	10240	0.046850	2622.84
20	20480	0.044010	5584.19
30	30720	0.043810	8414.52
40	40960	0.044470	11052.84
50	51200	0.088280	6959.67
60	61440	0.088330	8346.88
70	71680	0.088150	9757.91
80	81920	0.088060	11163.30
90	92160	0.088300	12524.58
100	102400	0.131900	9316.15
110	112640	0.131900	10247.76
120	122880	0.132200	11154.01
130	133120	0.132800	12028.92
140	143360	0.133100	12925.02
150	153600	0.175700	10490.61
160	163840	0.176600	11132.96
170	174080	0.177100	11795.37
180	184320	0.177400	12468.09
190	194560	0.177900	13123.78
200	204800	0.220900	11125.40
210	215040	0.221000	11676.38
220	225280	0.221600	12199.28
230	235520	0.221900	12736.55
240	**245760**	**0.221800**	**13296.30**
250	256000	0.265000	11592.45
260	266240	0.264900	12060.70
270	276480	0.265500	12496.27
280	286720	0.266500	12910.47
290	296960	0.309000	11532.43
300	307200	0.310300	11880.12
310	317440	0.310800	12256.37
320	327680	0.311700	12615.21
330	337920	0.312500	12976.13
340	348160	0.354800	11775.42
350	358400	0.355800	12087.69
360	368640	0.356000	12426.07
370	378880	0.356500	12753.32
380	389120	0.356400	13101.68
390	399360	0.399400	11998.80
400	409600	0.400600	12269.59
410	419840	0.400400	12582.62
420	430080	0.401200	12863.81
430	440320	0.401700	13153.70
440	450560	0.443400	12193.78
450	460800	0.444800	12431.65
460	471040	0.445400	12690.79
470	481280	0.446200	12943.43
480	491520	0.447600	13177.48
490	501760	0.489500	12300.55
500	512000	0.489500	12551.58

Table 1: Henon GPU performance (RTX 3070 Ti, lin15) with varying n_blocks (threads_per_block=1024, n_iter=1000000). Maximum GFLOPS (13296.302) achieved with n_blocks=240.

(d) **(5 points)** You should observe that

$$\frac{ExecutionTime}{\texttt{threads\_per\_block*n\_iter}}$$

is a stair step function of `n_blocks`. At what values of `n_blocks` to the steps occur? Why? Include a brief justification of your answer.
Hints:

- Think about scheduling blocks to run on the SMs of the GPU.
- The spacing of the steps is specific to the RTX 3070 Ti. Other GPUs would have their own spacings.

(e) **(5 points)** What is the speed-up relative to the host CPU? The `main` function reports the squared-step-size mean and standard deviation for both `henon_cpu` and `henon_gpu` so you can compare the result. Report the timing data that you get, report the speed-up, and make a few observations.

To get good speed-up measurements, it's best if the time interval between two calls to `getrusage` is at least 1 millisecond. On the other hand, because the `linXX` machines are in a shared lab, and CPSC 418 students usually `ssh` to them, it is good to keep your timing experiments to a maximum of 1 second to avoid causing problems for students working hard on their graphics homework. For some problems, the difference between the GPU and CPU speeds is large enough that its hard or impossible to satisfy these two guidelines for making good timing measurements when running the same problem on the GPU and the CPU. Our solution is that you find values for the parameters that maximize the throughput for each, with the constraints that run times should be between one millisecond and one second. We then take the speed-up as

$$\text{SpeedUp} \quad = \quad \frac{\text{GFlops}_{\text{GPU}}}{\text{GFlops}_{\text{CPU}}}$$

(f) **10 points)** Implement the double-precision versions, i.e. the `henon64` kernel and the `henon_cpu64` function for the host CPU.

Note: I don't like cut-and-paste code; so, I wrote a file `henon_help.h` that I could `#include` once to create the single-precision functions `henon32` and `henon_cpu32`, and a second time to create `henon64` and `henon_cpu64`. I preceded each `#include` with some `#define`s to make the first include create the single-precision versions and the second create the double precision versions. You don't have to do this, but if you don't like repeating code, feel free to do so.

(g) **(5 points)** What is the speed-up relative to the host CPU? The `main` function reports the squared-step-size mean and standard deviation for both `henon_cpu` and `henon_gpu` so you can compare the result. Report the timing data that you get, report the speed-up, and make a few observations.